cats.controller.ts JS

```
@Get()
async findAll(): Promise<any[]> {
  return [];
}
```

This code is perfectly valid. But Nest takes it a step further by allowing route handlers to return RxJS observable streams as well. Nest will handle the subscription internally and resolve the final emitted value once the stream completes.

cats.controller.ts JS

```
@Get()
findAll(): Observable<any[]> {
  return of([]);
}
```

****

```
GET /cats?age=2&breed=Persian
```

would result in `age` being `2` and `breed` being `Persian`.

If your application requires handling more complex query parameters, such as nested objects or arrays:

```
?filter[where][name]=John&filter[where][age]=30
?item[]=1&item[]=2
```

you'll need to configure your HTTP adapter (Express or Fastify) to use an appropriate query parser. In Express, you can use the `extended` parser, which allows for rich query objects:

```
const app = await NestFactory.create<NestExpressApplication>(AppModule);
app.set('query parser', 'extended');
```

****

In Fastify, you can use the `querystringParser` option:

```
const app = await NestFactory.create<NestFastifyApplication>(
  AppModule,
  new FastifyAdapter({
    querystringParser: (str) => qs.parse(str),
  }),
);
```

> **HINT**
> `qs` is a querystring parser that supports nesting and arrays. You can install it using `npm install qs`.

****

Scopes

Providers typically have a lifetime ("scope") that aligns with the application lifecycle. When the application is bootstrapped, each dependency must be resolved, meaning every provider gets instantiated. Similarly, when the application shuts down, all providers are destroyed. However, it's also possible to make a provider **request-scoped**, meaning its lifetime is tied to a specific request rather than the application's lifecycle. You can learn more about these techniques in the **Injection Scopes** chapter.

****

There are several ways to define a provider: you can use plain values, classes, and either asynchronous or synchronous factories.

****

Optional providers

Occasionally, you may have dependencies that don't always need to be resolved. For example, your class might depend on a **configuration object**, but if none is provided, default values should be used. In such cases, the dependency is considered optional, and the absence of the configuration provider should not result in an error.

To mark a provider as optional, use the `@Optional()` decorator in the constructor's signature.

```
import { Injectable, Optional, Inject } from '@nestjs/common';

@Injectable()
export class HttpService<T> {
  constructor(@Optional() @Inject('HTTP_OPTIONS') private httpClient: T) {}
}
```

In the example above, we're using a custom provider, which is why we include the `HTTP_OPTIONS` custom **token**. Previous examples demonstrated constructor-based injection, where a dependency is indicated through a class in the constructor. For more details on custom providers and how their associated tokens work, check out the Custom Providers chapter.

****

Property-based injection

The technique we've used so far is called constructor-based injection, where providers are injected through the constructor method. In certain specific cases, **property-based injection** can be useful. For example, if your top-level class depends on one or more providers, passing them all the way up through `super()` in sub-classes can become cumbersome. To avoid this, you can use the `@Inject()` decorator directly at the property level.

```
import { Injectable, Inject } from '@nestjs/common';

@Injectable()
export class HttpService<T> {
  @Inject('HTTP_OPTIONS')
  private readonly httpClient: T;
}
```

****

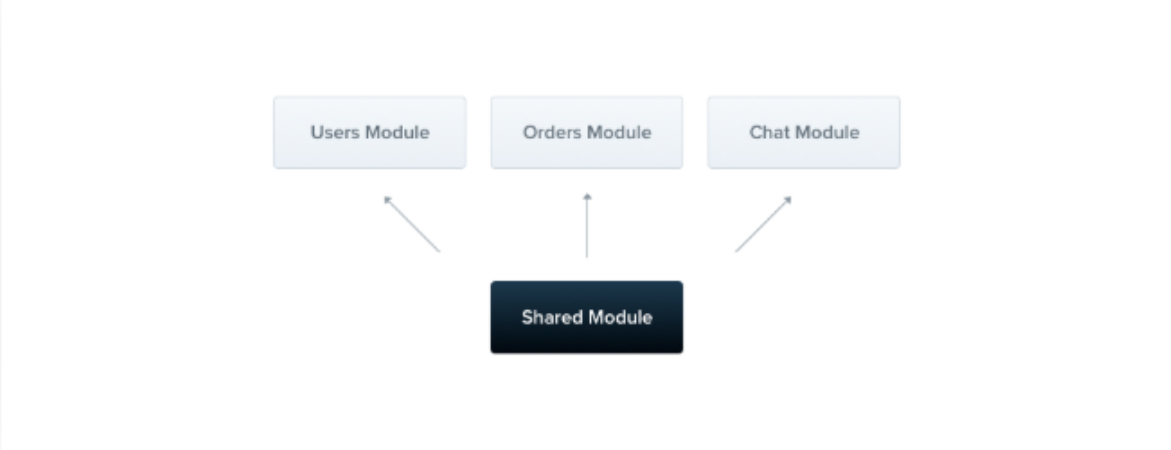
WARNING

If your class doesn't extend another class, it's generally better to use **constructor-based** injection. The constructor clearly specifies which dependencies are required, offering better visibility and making the code easier to understand compared to class properties annotated with `@Inject`.

****

Shared modules

In Nest, modules are **singletons** by default, and thus you can share the same instance of any provider between multiple modules effortlessly.



****

cats.module.ts JS

```
import { Module } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService]
})
export class CatsModule {}
```

Now any module that imports the `CatsModule` has access to the `CatsService` and will share the same instance with all other modules that import it as well.

****

Module re-exporting

As seen above, Modules can export their internal providers. In addition, they can re-export modules that they import. In the example below, the `CommonModule` is both imported into **and** exported from the `CoreModule`, making it available for other modules which import this one.

```
@Module({
  imports: [CommonModule],
  exports: [CommonModule],
})
export class CoreModule {}
```

****

Dependency injection

A module class can **inject** providers as well (e.g., for configuration purposes):

cats.module.ts JS

```
import { Module } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class CatsModule {
  constructor(private catsService: CatsService) {}
}
```

However, module classes themselves cannot be injected as providers due to circular dependency .

****

Global modules

If you have to import the same set of modules everywhere, it can get tedious. Unlike in Nest, Angular `providers` are registered in the global scope. Once defined, they're available everywhere. Nest, however, encapsulates providers inside the module scope. You aren't able to use a module's providers elsewhere without first importing the encapsulating module.

When you want to provide a set of providers which should be available everywhere out-of-the-box (e.g., helpers, database connections, etc.), make the module **global** with the `@Global()` decorator.

```
import { Module, Global } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Global()
@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}
```

****

The `@Global()` decorator makes the module global-scoped. Global modules should be registered **only once**, generally by the root or core module. In the above example, the `CatsService` provider will be ubiquitous, and modules that wish to inject the service will not need to import the `CatsModule` in their imports array.

> **HINT**
> Making everything global is not recommended as a design practice. While global modules can help reduce boilerplate, it's generally better to use the `imports` array to make a module's API available to other modules in a controlled and clear way. This approach provides better structure and maintainability, ensuring that only the necessary parts of the module are shared with others while avoiding unnecessary coupling between unrelated parts of the application.

****

Dynamic modules

Dynamic modules in Nest allow you to create modules that can be configured at runtime. This is especially useful when you need to provide flexible, customizable modules where the providers can be created based on certain options or configurations. Here's a brief overview of how **dynamic modules** work.

JS

```
import { Module, DynamicModule } from '@nestjs/common';
import { createDatabaseProviders } from './database.providers';
import { Connection } from './connection.provider';

@Module({
  providers: [Connection],
  exports: [Connection],
})
export class DatabaseModule {
  static forRoot(entities = [], options?): DynamicModule {
    const providers = createDatabaseProviders(options, entities);
    return {
      module: DatabaseModule,
      providers: providers,
      exports: providers,
    };
  }
}
```

> **HINT**
> The `forRoot()` method may return a dynamic module either synchronously or asynchronously (i.e., via a `Promise`).

This module defines the `Connection` provider by default (in the `@Module()` decorator metadata), but additionally - depending on the `entities` and `options` objects passed into the `forRoot()` method - exposes a collection of providers, for example, repositories. Note that the properties returned by the dynamic module **extend** (rather than override) the base module metadata defined in the `@Module()` decorator. That's how both the statically declared `Connection` provider **and** the dynamically generated repository providers are exported from the module.

If you want to register a dynamic module in the global scope, set the `global` property to `true`.

```
{
  global: true,
  module: DatabaseModule,
  providers: providers,
  exports: providers,
}
```

****

If you want to in turn re-export a dynamic module, you can omit the `forRoot()` method call in the exports array:

```
import { Module } from '@nestjs/common';
import { DatabaseModule } from './database/database.module';
import { User } from './users/entities/user.entity';

@Module({
  imports: [DatabaseModule.forRoot([User])],
  exports: [DatabaseModule],
})
export class AppModule {}
```

****

To generate middleware in nest: *nest g middleware name-here*

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class CatsMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log(`The request is: ${req.path}, with method: ${req.method}`)
    next()
  }
}
```

Then to apply the middleware, ex: in app.module.ts:

```
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { EventEmitterModule } from '@nestjs/event-emitter';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UsersModule } from './users/users.module';
import { CatsModule } from './cats/cats.module';
import { CatsMiddleware } from './cats/cats.middleware';

@Module({
  imports: [UsersModule, EventEmitterModule.forRoot(), CatsModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(CatsMiddleware).forRoutes('cats')
  }
}
```

And to apply to specific route and method:

```
import { MiddlewareConsumer, Module, NestModule, RequestMethod } from
'@nestjs/common';
import { EventEmitterModule } from '@nestjs/event-emitter';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UsersModule } from './users/users.module';
import { CatsModule } from './cats/cats.module';
import { CatsMiddleware } from './cats/cats.middleware';

@Module({
  imports: [UsersModule, EventEmitterModule.forRoot(), CatsModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(CatsMiddleware).forRoutes({ path: 'cats', method:
RequestMethod.GET })
  }
}
```

HINT

The configure() method can be made asynchronous using async/await (e.g., you can await completion of an asynchronous operation inside the configure() method body).

****

abcd/ with no additional characters will not match the route. For this, you need to wrap the wildcard in braces to make it optional

```
forRoutes({
  path: 'abcd/{*splat}',
  method: RequestMethod.ALL,
});
```

****